

Question 2

(a) **value_iteration_a.py**- Implements Value Iteration algorithm.

1. Initialize:

$$V(s) = 0$$

2. Iteratively update:

Compute expected future value

Compute Q-function:

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} P((s, a), s') V(s')$$

Update value:

$$V(s) = \max_a Q(s, a)$$

3. Stop when:

$$\max_s |V_{new}(s) - V(s)| < \epsilon$$

(b) **induced_policy_b.py**- Extracts greedy policy from Q-function:

Selects actions with maximum Q-value

Breaks ties uniformly

(c) **policy_iteration_c.py**- Initialize policy

Start with uniform random policy

Policy Evaluation

Compute value function:

$$V^\pi(s) = R^\pi(s) + \gamma \sum_{s'} P^\pi(s, s') V^\pi(s')$$

Policy Improvement

Update policy: $\pi_{new}(s) = \arg \max_a Q(s, a)$

Repeat the process until policy converges.

(d) **run_experiment_q2.py**- We compare value functions obtained from:

- Value Iteration: V_{VI}
- Policy Iteration: V_{PI}

L1 Difference $L1 = \sum_s |V_{VI}(s) - V_{PI}(s)|$

Observation- Ideally: $V_{VI} \approx V_{PI}$

Both methods converge to **same optimal solution**

(e) Runtime Analysis-

Principle- Measure execution time for:

- Value Iteration
- Policy Iteration

Working

For different grid sizes N :

1. Compute optimal Q using VI
2. Compute optimal Q using PI
3. Record execution time

Observations

Value Iteration- Involves many iterations, but it is cheap per iteration.

Policy Iteration- Involves fewer iterations, but expensive per step.

